ТЕСТОВОЕ ЗАДАНИЕ

Вредоносные APK и байтовые сигнатуры

Задание 1 — подборка хеш-сумм вредоносных APK с вердиктами вендоров.
Задание 2 — прототип генератора байтовых сигнатур по группам файлов G1 и G2.

Выполнил Белов Т.Ю. (student367, tg: @ares_alive)

Дата 06.11.2026

Содержание

01 **Ключевые выводы** 3

02 **Задание 1. Подборка вредоносных APK** 4

Методика сбора 4

Схема 1. Сбор подборки 5

Семейства и покрытие вердиктами 6

Образцы с наибольшим числом детектов 7

Вредоносные файлы внутри APK 7

03 **Задание 2. Генератор байтовых сигнатур** 9

Постановка и алгоритм 9

Схема 2. Алгоритм sigtool 10

Использование 11

Проверка: тесты и демонстрация 11

Ограничения прототипа 12

Приложение А. Подборка хеш-сумм 13

Приложение Б. Состав материалов 16

Приложение В. Использование ИИ 17

01
Ключевые выводы

Задание 1. Собрана подборка из **2000 SHA-256 вредоносных APK** 58 семейств с **16 772 вердиктами** вендоров. Образцы не скачивались: всё получено из отчётов MalwareBazaar и VirusTotal через API.

- **Вердикты.** У 1996 образцов есть вердикт хотя бы одного вендора: ReversingLabs, Triage, CERT-PL MWDB, FileScan-IO, Spamhaus, InQuest, YOROI, Kaspersky OpenTIP. Для 137 образцов дополнительно собраны детекты ~70 AV-движков VirusTotal (медиана 32 детекта, максимум 42 из 70).
- **Состав.** Основа подборки — банковские трояны и RAT: IRATA, SpyNote, Cerberus, Hydra, Coper, Triada, Alien, Joker. Четыре крупнейших семейства дают треть подборки.
- **Бонус — файлы внутри APK.** В 112 APK найдено 196 вредоносных вложенных файлов. Чаще всего детектируется основной `classes.dex` (74 APK). В 18 APK вредоносная часть вынесена в нативные библиотеки `lib*.so`: Mirai-подобные боты, распаковщики дропперов, майнер.

Задание 2. Реализован прототип `sigtool`. На вход он получает группы файлов G1 и G2, на выходе даёт байтовые массивы: каждый файл G1 содержит хотя бы один из них, ни один файл G2 не содержит ни одного. Кандидаты ищутся по хешам k-грамм с winnowing, сигнатуры выбираются жадным покрытием множества и затем расширяются до 64 байт. APK раскрываются до записей ZIP, результат выводится в text, JSON или YARA. Все 8 тестов проходят; на демонстрационном наборе одна сигнатура покрывает 5 из 5 файлов G1 и не встречается в 10 файлах G2.

ВОПРОС ЗАДАНИЯ	ГДЕ ОТВЕТ
1. Подборка хеш-сумм вредоносных APK	Приложение А; полный список — `apk_verdicts.csv`
1. Перечень вердиктов различных вендоров	Раздел 02, «Семейства и покрытие вердиктами», «Образцы с наибольшим числом детектов»
1. Бонус: вредоносные файлы внутри APK	Раздел 02, «Вредоносные файлы внутри APK»
2. Прототип инструмента G1 / G2	Раздел 03, схема 2
Прочее	Ограничения прототипа, состав материалов

02
Задание 1. Подборка вредоносных APK

Задача	Подборка хеш-сумм вредоносных APK с перечнем вердиктов различных вендоров; бонус — конкретные вредоносные файлы внутри APK
Объём	**2000** APK · 58 семейств · 16 772 вердикта (12 762 от вендоров MalwareBazaar, 4010 от движков VirusTotal)
Период	Первое появление образцов в MalwareBazaar: 22.04.2020 – 06.11.2026
Источники	MalwareBazaar API (abuse.ch), VirusTotal API v3; образцы не скачивались
Инструмент	`task1_apk_hashes/collect.py` — Python 3, только стандартная библиотека
Результат	`apk_verdicts.csv`, `apk_verdicts_long.csv`, `apk_inner_files.csv`, `apk_verdicts.json`

Методика сбора

1. **Список образцов.** Из MalwareBazaar запрашиваются последние 1000 APK (`get_file_type`), выборки по 48 Android-семействам (`get_siginfo`) и 5 тегам (`get_taginfo`). Списки объединяются, дубликаты по SHA-256 удаляются, остаются записи с `file_type = apk` .
2. **Вердикты MalwareBazaar.** Для каждого образца запрашивается `get_info` : блок `vendor_intel` с вердиктами внешних вендоров и сработавшие YARA-правила.
3. **Порядок обхода.** Образцы упорядочиваются по семействам по кругу (round-robin). Так ограниченная квота VirusTotal расходуется на разные семейства, а не на одно крупное.
4. **Вердикты VirusTotal.** Запрос `/files/{sha256}` : детекты AV-движков, `popular_threat_classification` , Androguard-метаданные (пакет, опасные разрешения).
5. **Файлы внутри APK.** Если у образца есть хотя бы один детект, запрашивается `/files/{sha256}/bundled_files` (до 40 вложенных файлов). Отбираются файлы с `malicious > 0` .
6. **Выгрузка.** CSV (образец на строку и длинный формат «образец — вендор — вердикт»), JSON и Markdown-сводка.

> **Ограничение квоты.** Публичный ключ VirusTotal даёт 4 запроса в минуту и 500 в сутки. Скрипт выдерживает паузу 15,5 с, повторяет запрос при HTTP 429 и останавливает опрос при `QuotaExceededError` . Все ответы кэшируются в `cache/` : повторный запуск не тратит квоту, а `--report-only` пересобирает выгрузку офлайн. Поэтому детекты VirusTotal собраны для 137 образцов, а вердикты MalwareBazaar — для всей подборки.

Схема 1. Сбор подборки

обработка · запросы к VirusTotal / по каждому образцу · вердикты о вредоносности · служебные шаги

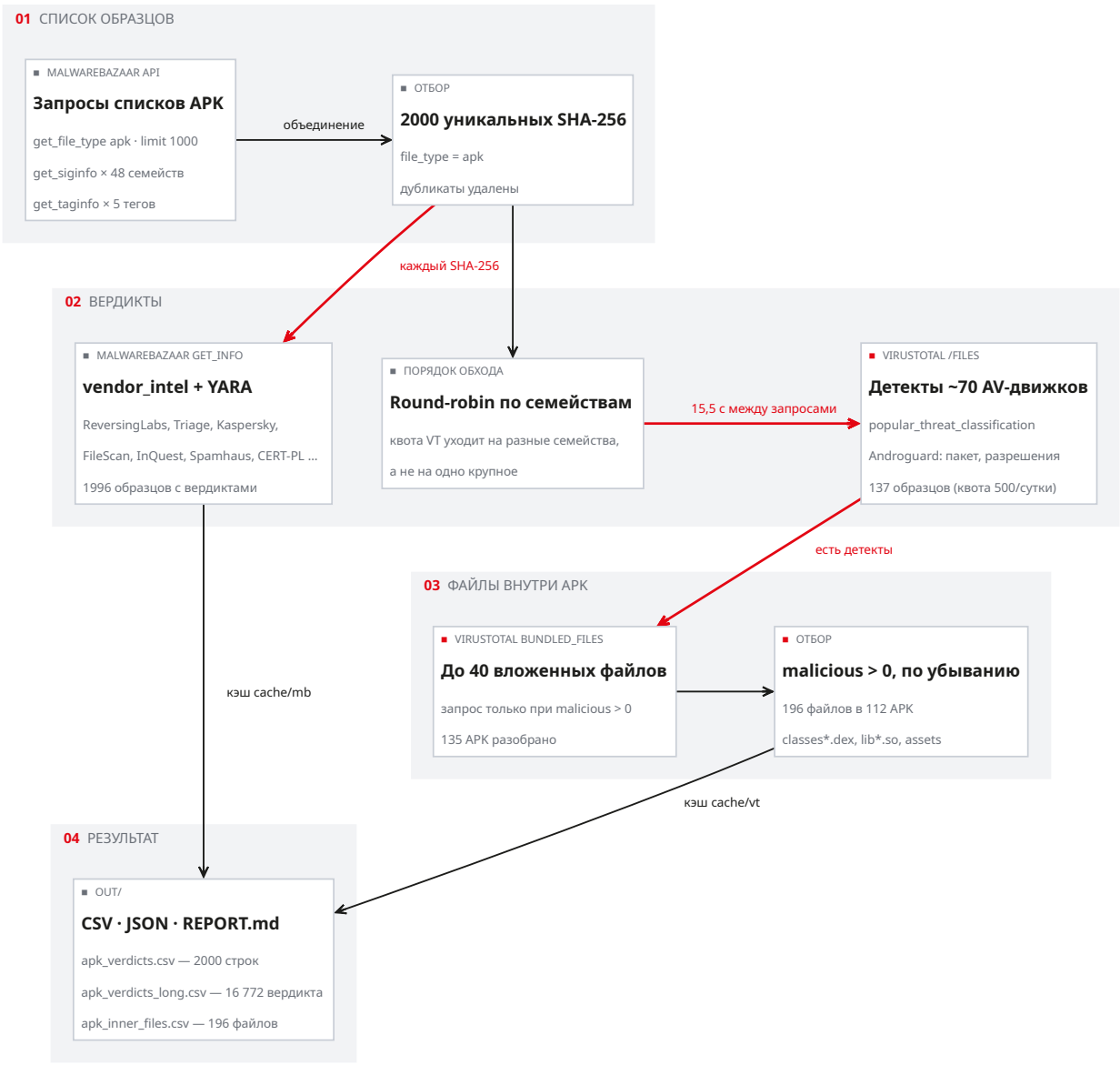
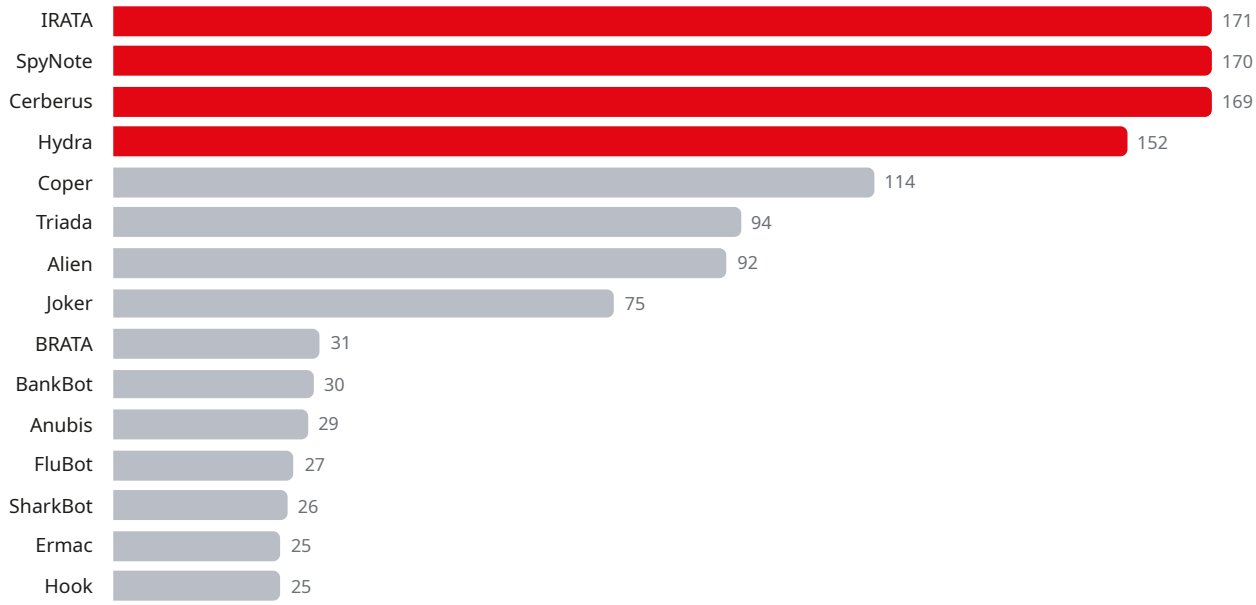


Схема 1. От списков MalwareBazaar до выгрузки с вердиктами и вредоносными файлами внутри APK.

Семейства и покрытие вердиктами

Четыре семейства — IRATA, SpyNote, Cerberus, Hydra — дают треть подборки

Топ-15 из 58 семейств по MalwareBazaar: 1230 из 2000 образцов; ещё 608 образцов без метки семейства



Вердикт хотя бы одного вендора есть у 1996 из 2000 образцов. Покрытие по источникам:

ИСТОЧНИК ВЕРДИКТОВ	ОБРАЗЦОВ	ЧТО СОДЕРЖИТ ВЕРДИКТ
ReversingLabs	1996	Имя угрозы, статус (MALICIOUS 1227 · KNOWN 420 · SUSPICIOUS 349), доля сработавших сканеров
Triage	1963	Семейство и оценка опасности 1–10
CERT-PL MWDB	1735	Семейство
FileScan-IO	1700	MALICIOUS / LIKELY_MALICIOUS / SUSPICIOUS
Spamhaus HBL	1543	malicious / suspicious
InQuest	1525	Вердикт
YOROI YOMI	1497	Вердикт
Kaspersky	794	Вердикт Kaspersky OpenTIP
YARA (MalwareBazaar)	1021	Имена сработавших правил
VirusTotal	**137**	**Детект каждого из 64 сработавших движков; медиана 32, максимум 42 детектов**

Чаще всего на образцы с отчётом VirusTotal срабатывали ESET-NOD32 (128 из 137), BitDefenderFalx (127), SymantecMobileInsight и Tencent (по 126), K7GW (125), Avast-Mobile (123), Kaspersky (122), DrWeb (121).

Образцы с наибольшим числом детектов

SHA-256	СЕМЕЙСТВО	VT	KASPERSKY	ESET-NOD32	ФАЙЛЫ ВНУТРИ (ДЕТЕКТОВ)
`a15e50a75302…`	JackSkid	42/68	HEUR:Backdoor.Linux.Mirai.ld	Android/Agent.FSQ trojan	`lol` (35) `classes.dex` (6)
`ae5483625f27…`	KATARU	42/66	HEUR:Exploit.Linux.CVE-2026-31431.c	Android/Agent.FVD trojan	`libbot.so` (20) `libbot.so` (20)
`ca3da13d69ce…`	Octo	42/70	UDS:Trojan-Dropper.Linux.Agent.gen	a variant of Android/TrojanDropper.Agent.LWH	`libTCqnfQ.so` (32) `classes.dex` (12)
`724a56172f40…`	Sova	40/63	HEUR:Trojan-Banker.AndroidOS.Sova.i	multiple detections	`9ae5523ea2…` (25) `classes.dex` (23)
`1a9a54eb8b3e…`	JackSkid	40/67	HEUR:Backdoor.Linux.Mirai.ld	Android/Agent.FSQ trojan	`lol` (35) `lol2` (33)
`95a1793b5bd3…`	trojan.basdoor/spyagent	40/64	HEUR:Backdoor.AndroidOS.Basdoor.c	a variant of Android/Spy.Agent.BWC	`classes.dex` (24)
`a778818cc08d…`	FluBot	39/67	HEUR:Trojan-Banker.AndroidOS.Bian.f	Android/TrojanDropper.Agent.ICZ trojan	`classes.dex` (20)
`47bb7f855cdf…`	Teabot	39/66	HEUR:Trojan-Banker.AndroidOS.Agent.io	Android/TrojanDropper.Agent.IVA trojan	`classes.dex` (20) `AndroidManifest.xml` (6)
`153410238d01…`	Chameleon	39/68	HEUR:Trojan-Dropper.AndroidOS.Hqwar.ia	Android/TrojanDropper.Agent.LCK trojan	`classes.dex` (23)
`376d13affcbf…`	Sova	39/66	HEUR:Trojan-Dropper.AndroidOS.Hqwar.hr	Android/TrojanDropper.Agent.LAU trojan	`classes.dex` (23) `classes2.dex` (1)

Полные SHA-256 — в приложении А; все вердикты всех вендоров по каждому образцу — в `apk_verdicts_long.csv`.

Вредоносные файлы внутри APK

По `bundled_files` разобрано 135 APK. В 112 из них найдено **196 вредоносных вложенных файлов**. По типам VirusTotal: Android — 157, ELF — 30, JavaScript — 3, Win32 DLL — 2, unknown — 2, ZIP — 1, HTML — 1.

ВЛОЖЕННЫЙ ФАЙЛ	APK	ЧТО ЭТО ОЗНАЧАЕТ
`classes.dex`	74	Вредоносный код в основном байт-коде приложения
`AndroidManifest.xml`	17	Детект по набору разрешений и компонентов в манифесте
`classes2.dex` … `classes5.dex`	27	Нагрузка вынесена в дополнительные DEX (multidex)
`libbot.so`, `libattack.so`, `libTCqnfQ.so`, `libMJnWUXJ.so`, `xmrutil.so`	**18**	**Нативные ELF: Mirai-подобные боты, распаковщики дропперов (Coper/Octo), майнер**
`lol`, `lol2`, `1.bt`	**5**	**ELF-нагрузка без расширения (JackSkid / Mirai)**
Вложенные APK и архивы: `Банковский троян.apk`, `Mellat.zip`	2	Дропперы, несущие вторую стадию
`resources.arsc`, `index.html`, `config.json`	6	Вероятно, фишинговые страницы и конфигурация; по именам и типам, вручную не проверялось

Вложенные файлы с наибольшим числом детектов

APK (SHA-256)	ФАЙЛ ВНУТРИ	ТИП	ДЕТЕКТОВ	ПРИМЕР ВЕРДИКТА
`1f333d910158…`	`xmrutil.so`	ELF	38	Lionic: Riskware.Linux.Camelot.1!c
`a15e50a75302…`	`lol`	ELF	35	Lionic: Trojan.Linux.Mirai.K!c
`1a9a54eb8b3e…`	`lol`	ELF	35	Lionic: Trojan.Linux.Mirai.K!c
`1a9a54eb8b3e…`	`lol2`	ELF	33	Lionic: Trojan.Linux.Mirai.K!c
`ca3da13d69ce…`	`libTCqnfQ.so`	ELF	32	Lionic: Trojan.Linux.Coper.7!c
`d62705186c48…`	`classes.dex`	Android	32	Lionic: Trojan.AndroidOS.AndroRAT.C!c
`2652bd208ccd…`	`dff5691e133f…`	Android	32	Lionic: Trojan.AndroidOS.Agent.C!c
`0c485d9cbf8a…`	`classes.dex`	Android	31	Lionic: Trojan.AndroidOS.Ahmyth.C!c
`77ffe8b7b3ef…`	`classes.dex`	Android	30	Lionic: Trojan.AndroidOS.Metasploit.C!c
`8e39fb18e8ac…`	`557c02b5e9ca…`	Android	30	Lionic: Trojan.AndroidOS.Agent.C!c

Вывод. В большинстве APK вредоносная логика находится в DEX-коде. Но в части семейств (Coper/Octo, JackSkid, KATARU) основной детект приходится на нативную библиотеку или ELF в `assets`, а сам `classes.dex` детектируется слабо. Полный перечень с SHA-256 вложенных файлов — в `apk_inner_files.csv`.

03
Задание 2. Генератор байтовых сигнатур

Задача	По двум группам файлов G1 и G2 выдать байтовые массивы, полностью покрывающие файлы G1 и отсутствующие в файлах G2
Инструмент	`task2_sigtool/sigtool.py` — Python 3 + NumPy, 410 строк
Вход	Файлы и каталоги G1 и G2; с `--unpack` — записи внутри APK/ZIP/JAR
Выход	Набор сигнатур (hex, ASCII, места вхождения) в формате text, JSON или YARA; список непокрытых файлов
Проверка	8 из 8 модульных тестов; демонстрация на синтетических APK

Постановка и алгоритм

Нужно найти набор массивов S, для которого выполняются два условия:

$$\forall f \in G1\ \exists s \in S: s \subset f \quad \text{и} \quad \forall g \in G2\ \forall s \in S: s \not\subset g$$

Это задача покрытия множества (set cover): кандидаты — подстроки файлов G1, которых нет в G2, покрываемые элементы — файлы G1. Точное решение NP-трудно, поэтому кандидаты быстро ищутся по хешам, а сигнатуры выбираются жадно.

1. **Хеши k-грамм.** Каждый файл режется на k-граммы (k = 12). Для каждой векторно в NumPy считается 64-битный полиномиальный хеш с перемешиванием splitmix64.
2. **Winnowing.** Из хешей G1 остаётся минимум в каждом окне w = 32. Любая общая подстрока длиной от w + k − 1 = 43 байт гарантированно даёт общий отпечаток, а кандидатов становится примерно в (w + 1) / 2 ≈ 16 раз меньше.
3. **Фильтр по G2.** Все k-граммы каждого файла G2 (не только отпечатки) ищутся среди кандидатов бинарным поиском; совпавшие удаляются. Файлы G2 читаются потоком и не хранятся в памяти.
4. **Матрица покрытия.** Строится булева матрица «кандидат × файл G1». Кандидаты, где меньше 6 различных байт, отбрасываются как паддинг и таблицы.
5. **Жадное покрытие.** Берётся кандидат, покрывающий больше всего ещё не покрытых файлов; при равенстве — с большим числом различных байт. Жадный алгоритм даёт приближение ln |G1| к оптимуму. `--redundancy N` требует N разных сигнатур на файл.
6. **Расширение.** Выбранная k-грамма бинарным поиском расширяется влево и вправо до 64 байт, пока массив есть во всех покрываемых файлах. Надстрока строки, которой нет в G2, тоже отсутствует в G2, поэтому повторный фильтр не нужен.
7. **Проверка.** Независимая побайтовая проверка исключает ложные совпадения из-за коллизий хешей. При ошибке код возврата 2.

Файл G1, который целиком встречается в G2, покрыть невозможно. Он попадает в список непокрытых, код возврата — 1. С `--unpack` инструмент рекурсивно (глубина 2) раскрывает APK, JAR и вложенные APK. Это важно: сжатые deflate данные разных APK различаются даже при одинаковом `classes.dex`. Место сигнатуры выводится как `файл.apk!classes.dex@0x5a23`.

Схема 2. Алгоритм sigtool



Схема 2. Семь шагов от групп файлов до проверенных сигнатур.

Использование

```
python3 sigtool.py --g1 samples/family/ --g2 samples/clean/ --unpack -f yara -o rule.yar
```

КЛЮЧ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`--g1`, `--g2`	—	Файлы или каталоги групп (каталоги обходятся рекурсивно)
`-k`, `--min-len`	12	Длина k-граммы = минимальная длина сигнатуры
`-w`, `--window`	32	Окно winnowing; 1 = полный перебор
`--max-len`	64	Максимальная длина сигнатуры
`-r`, `--redundancy`	1	Сколько разных сигнатур должно попадать в каждый файл G1
`--min-distinct`	6	Минимум различных байт в k-грамме (отсев паддинга)
`--require-all`	выкл.	Только массивы, общие для всех файлов G1
`--unpack [DEPTH]`	выкл. (глубина 2)	Раскрывать ZIP/APK/JAR
`-f`, `--format`	text	text, json или yara
`--no-verify`	выкл.	Пропустить финальную побайтовую проверку

Коды возврата: 0 — все файлы G1 покрыты, 1 — есть непокрытые, 2 — ошибка финальной проверки.

Проверка: тесты и демонстрация

`python3 -m unittest -v test_sigtool` — 8 из 8 тестов пройдены за 0,05 с. В каждом тесте дополнительно проверяется контракт: покрытие G1 и отсутствие сигнатур в G2.

ТЕСТ	СЦЕНАРИЙ	ОЖИДАЕМЫЙ РЕЗУЛЬТАТ
`common_marker_single_signature`	5 файлов G1 с общим маркером и блок, общий с G2	1 сигнатура длиной 64, не лежит целиком в блоке G2
`two_subfamilies_need_two_signatures`	Две подгруппы с разными маркерами	Ровно 2 сигнатуры
`require_all_mode`	Нет массива, общего для всех G1	Пустой результат, оба файла не покрыты
`file_fully_inside_g2_is_uncovered`	Файл G1 целиком внутри файла G2	Именно он в списке непокрытых
`padding_is_not_a_signature`	500 нулевых байт во всех G1	Паддинг не становится сигнатурой
`redundancy`	`-r 2`, два маркера в каждом файле	В каждый файл попадают 2 сигнатуры
`unpack_apk_finds_payload_in_inner_dex`	APK с «фреймворком», общим с чистыми APK, и нагрузкой в assets	Сигнатура из `payload.bin`, а не из фреймворка
`cli_yara_output_compiles_and_matches`	Вывод `-f yara` через CLI	Правило компилируется `yara`: 3 срабатывания в G1, 0 в G2

Демонстрация на синтетических APK

Набор: 5 «вредоносных» APK (G1) и 10 «чистых» (G2). У всех в `classes.dex` общий фрагмент 20 КБ, в G1 после него идёт блок нагрузки с URL «C2» (смещение 0x59d8).

```
$ python3 sigtool.py --g1 demo/g1 --g2 demo/g2 --unpack
G1: 5 файлов, G2: 10 файлов
[1] отпечатков G1: 2491
[2] после фильтрации G2: 1300
[3] информативных кандидатов: 1300
[4] выбрано сигнатур: 1
[5] проверка пройдена: 1 сигнатур, покрыто 5/5 файлов G1, 0 совпадений в G2
sig0  len=64  covers=5/5
  hex:   9d dc 60 1a b1 de c2 8e ae a9 37 b7 f7 c9 e8 52 6f 1b ed 3a fe 85 58 7d …
    - demo/g1/mal0.apk!classes.dex@0x5a23
    … mal1 — mal4: то же смещение
```

Сигнатура найдена внутри блока нагрузки (0x5a23 > 0x59d8); общий с G2 фрагмент отброшен. Тот же результат в формате YARA:

```
rule sigtool_generated
{
  meta:
    generator = "sigtool"
    g1_files = 5
    note = "signatures were extracted from UNPACKED zip/apk entries; scan extracted files, not the compressed
container"
  strings:
    $s0 = { 9d dc 60 1a b1 de c2 8e ae a9 37 b7 f7 c9 e8 52 6f 1b ed 3a fe 85 58 7d 30 88 3e 2e 7d 71 24
            49 3c 07 bb b3 04 6e 9c 36 68 fc b5 67 42 66 7a 7b 36 24 04 1a dd 52 5d c3 4b f6 ef 6d 5e 66 8a 38 }
// covers 5
  condition:
    any of them
}
```

Ограничения прототипа

- **Не минимальный набор.** Жадное покрытие не гарантирует минимального числа сигнатур; точная задача NP-трудна.
- **Короткие общие фрагменты.** При w > 1 фрагменты короче w + k − 1 = 43 байт могут быть пропущены; `-w 1` включает полный перебор ценой памяти.
- **Точные байты.** Сигнатуры без wildcard неустойчивы к перекомпиляции и обфускации; семантика DEX (классы, методы, строки) не учитывается.
- **Зависимость от G2.** Если в G2 нет типичных библиотек (AndroidX, OkHttp), сигнатура может попасть в них и давать ложные срабатывания.
- **Память.** Файлы G1 целиком загружаются в память; для сотен крупных APK нужен соответствующий объём RAM.
- **Сжатые контейнеры.** С `--unpack` сигнатуры берутся из распакованных записей, поэтому сканировать нужно извлечённые файлы, а не сжатый APK. Это указано в `meta` YARA-правила.

Приложение А. Подборка хеш-сумм

В отчёт вынесены две выборки; все 2000 образцов со всеми вердиктами — в `apk_verdicts.csv` и `apk_verdicts_long.csv`.

А.1. По одному образцу на семейство (53 семейства)

Вердикты ReversingLabs и Triage из MalwareBazaar; рядом с именем угрозы — доля сработавших сканеров ReversingLabs.

SHA-256	СЕМЕЙСТВО	REVERSINGLABS	TRIAGE
bfc577ea1b9b326609d857e0c7ea4c7db5c382f86b3faeb92e9230da650b3af9	AhMyth	Android.Trojan.Ahmyth 15/38	ahmyth score=10
8b5cfe53c6b30897bed06d794d73c0bb499620ca1e0128f18b1ae91a197d8548	Ajina	Android.Trojan.Generic 7/35	ajina score=10
3269a0116ce0915afdd32e8d87b05ae9f5c00fcf0a6729add39a6a87aaf30089	Albiriox	Android.Trojan.Multiverze 5/37	score=7
39ccc5d7009aefac18891c70a16b857b98481d6f69800075e31900c4dc9ef000	Alien	Android.Dropper.Hqwar 9/25	alienbot score=10
d62705186c488bb26fccdb1404931223a887004fd6704ac1483e599a15e92792	AndroRAT	Android.Trojan.AndroRAT 24/43	score=8
392272ef515d2f60f2c058675d637bf63a265800b8e4613ed9f72eeb8ebb323d	Anubis	Android.Infostealer.Anubis 10/38	score=6
cadcce6d4088c72fd0779d59d00f5753eaab585a263e3c92374df29ae767ac15	BankBot	Android.Trojan.Coper 13/37	score=6
1faeeec49012ef73fbe7fabcb325b0b849f64a967c9d0d777c80da9870a34b70	BingoMod	KNOWN 4/38	bingomod score=10
a25bf4bdb2ed9872456af0057eb21ce31fd03d680d63a9da469519060b4814bc	BlackRock	Android.Spyware.Flubot 9/23	score=8
1430dc0cc26adfb7b7715624602767af26f453e3f34df66f1a6d41c38a35819c	BRATA	Android.Trojan.SpyNote 15/41	brata score=10
1f333d910158316c6211f7b24d118aadca778abb3964f3a4b7d3914e49278b7d	BtmobRAT	Linux.Trojan.BitCoinMi 24/36	btmobrat score=10
2152b98a832ce3e442b486ceeaa8e41a8a99657168986757913e2d916c7b1c21	CECbot	Linux.Worm.Mirai 18/38	cecbot score=10
8beae1f6b21cec17eed82fb7af25e1782d5b7bf10fd22369603313f1b1a5e5e4	Cerberus	Android.Infostealer.Anubis 10/38	cerberus score=10
153410238d01773e5c705c6d18955793bd61cb2e82c5c7656e74563bb43b3ffa	Chameleon	Android.Trojan.Generic 7/37	score=7
0628ad6d1fd836b13b22e75fa169502d8ce78b7ad20f0261eb5151da98437bca	CoinMiner	Linux.Coinminer.Generic 16/24	xmrig score=10
dc2a780f6abb9f0ec0a6675f20acb91ebe2a8748297682a59daf9164fbea2ee8	Coper	Android.Dropper.Generic 14/38	octo score=10
464fee5a6d85370e8764f0e682ef01cf2d9cef3efb7e4bbdf0146a94cc83ff4a	Copybara	KNOWN 5/36	copybara score=10
5f94e59c132a7ed0e35b60146d973a728ab9f390c8291f9547cee26c56427738	CraxsRAT	KNOWN 2/38	
b9ddc7c5f9adbf012494ab258acd80ab35311b52e3b28f2a78a1d8dad75ab259	Crocodilus	Android.Dropper.Generic 7/23	
14b005352c07490b3689ac4eb187eb68d2b50cd135e57e1bca2f3469b10e2dd5	Ermac	Android.Trojan.Ermac 10/23	
a778818cc08d213d1f66db5fa8b34aabc862569ecb99fd1aea37ff6ccab48400	FluBot	Android.Trojan.BankBot 10/37	flubot score=10
45f232ed48389196190252e0abb31606422d0894ff48c593571662579b087686	Gentlemen	KNOWN 1/36	gentlemen score=10
a6ea793e52823218041ededc61900c6ea273b50ec64d32c4d2a3ce722450705b	GhostGrab	Android.Packed.Generic 7/24	ghostgrab score=10
173192ec576111adbc0c228f75244cf627513c95ca1709036e9850f0cdb7c90b	GhostRAT	Android.Backdoor.GhostRAT 10/36	nfcshare score=10
a6ed100ae42e4fdabfd1b4c992762152bc4a11cc8e521b647b444c75bb7a9782	GodFather	Android.Infostealer.Banbra 11/36	godfather score=10
29926c0101734a0c34efd310d58d055f266e28960c65f00f905327dcbb186711	Harly	KNOWN 2/38	score=7
e084a327fde5523b1803040fc1ea40be84bc8434913d37d01083890c351dec96	Herodotus	Android.Trojan.Generic 5/24	score=7
6ca691c97ab0028ce9acade5d079289668b1d0339ad330181c84fc7e7114c509	Hook	Android.Trojan.Multiverze 8/24	hook score=10
d300e348a236c6fc23bceae84fdbd6deecf8f41565c032c0d062733fcb066a6e	Hydra	Android.Trojan.Ravartar 14/38	hydra score=10
425469176c41994bc33ce0582948798875f5c9682b7d0967c925a10f813e1a3d	IRATA	Android.Trojan.Generic 9/24	irata score=10
a15e50a753028ff9db51885705d6348113dac5b653cd2bd00dcfd8b97da7566a	JackSkid	Linux.Worm.Mirai 12/24	score=6
2fa0f3893ca8fff0ba2c2e30d0ba35712ec5dffa27ebe1b87b4a26cb4f7d0aa4	Joker	KNOWN 1/36	score=6

ae5483625f278378e9d8930562c3be0dc41c9132fbcaad3894c78b76eb986141	KATARU	Linux.Exploit.CVE-2026-43284 16/23	mirai score=10
77ffe8b7b3efa39fa35daaee1f70f560079c96c680f9d9d85997d658e2b367a9	Metasploit	Android.Hacktool.MetaSploit 14/24	metasploit score=10
c48071a9e15df2cad4c027a92370aab2f0187eec5438977c47463d505f271740	Mirai	Linux.Worm.Mirai 17/36	score=6
3613a76be3c6f0d7f6108920cbd4bcedc4c9a4c3607136a747f77f3c099f7322	Monokle	Android.Trojan.Multiverze 5/38	score=1
af56cbeba5bb0e5b10af555f7380214d89f8f1bfcca229f2e713c7eb4a371630	NGate	KNOWN 0/38	ngate score=10
ca3da13d69ce07ed9b31314f2404f9c6a2235067406b545d7b00517258cee9d6	Octo	Android.Trojan.Generic 19/38	octo score=10
1fa2150192384a7abb27ad92295aa937efca1b2ac88dc802d3a68082d61c38e9	PhantomCard	Android.Trojan.Multiverze 13/38	phantomcard score=10
557c02b5e9ca8103e6ad5413df1b447fae5c9ce086a0b5330bf9c79a578585df	PixRevolution	Android.Trojan.Multiverze 14/38	pixrevolution score=10
f6bf91fd210cf04592949131b3ed1a4a10986c9bdeebb4869f511b42c54c5328	RatHat	Android.Hacktool.Multiverze 10/23	rathat score=10
e7ebac9ba2548db5e772f1a0d657acf0f61425e67cf326bea142a1f1c13a077d	RedHook	Android.Spyware.Multiverze 6/21	redhook score=10
71c78101f7792fe879a082e323fed89c5e4a43132d01d3f79ed02afd8db45497	SharkBot	Android.Spyware.Sharkbot 16/25	sharkbot score=10
1ae3e8213b07e1d53eca54a1901e2dc6dc9e27a3ca10377a63e6adec2fe15426	SilentNet	KNOWN 4/23	score=3
724a56172f40177da76242ee169ac336b63d5df85889368d1531f593b658606b	Sova	Android.Trojan.Multiverze 14/24	sova score=10
3745e0fb7edbdd8da57f44f5ee1d2b2cc0db5d7f8b63ea69ce12ce561402cf17	SpinOK	Android.Spyware.SpinOK 8/37	spinok score=10
cde47bed5ec83e29f6455ec35228c3f56d488393668f5b720b79fcd6cb3bc098	SpyMax	Android.Trojan.Phonzy 12/38	spymax score=10
d472e114361dd2c6ebafb60daa72ba2db09752de5b243538f0fee18410ad6a25	SpyNote	Android.Trojan.AVerseFalc 7/24	ghostbat score=10
47bb7f855cdf116c62499240089fa1b7a69585e8b7f639e192b9d038da4094cd	Teabot	Android.Infostealer.Generic 8/33	teabot score=10
fde931224d2e558e67ac8c9c0c1d0aac4f7562622a67870d6c3024bdeb851676	TgToxic	Android.Trojan.ScarletFlash 8/24	tgtoxic score=10
41752e7b3d8374ec74b94ebc258a0fefd3d41a44bb07a43d85546701afb50e36	TheTruthSpy	Android.Spyware.Nidb 12/24	truthspy score=10
de7d54276047fc457271b582a378e9599ce0b3822d4e9cb89a1260f81a3c2644	Triada	Android.Trojan.Alevaul 5/22	triada score=10
f4d7e9ec4eda034c29b8d73d4790846588858f56e67909c2ffedf9223d7ca9bd2	Vultur	Android.Spyware.Vultur 14/24	vultur score=10

A.2. 40 образцов с наибольшим числом детектов VirusTotal

SHA-256	VT	KASPERSKY	DRWEB
a15e50a753028ff9db51885705d6348113dac5b653cd2bd00dcfd8b97da7566a	42/68	HEUR:Backdoor.Linux.Mirai.ld	Android.Siggen.Susp2.4485
ae5483625f278378e9d8930562c3be0dc41c9132fbcaad3894c78b76eb986141	42/66	HEUR:Exploit.Linux.CVE-2026-31431.c	Android.Backdoor.Kyorij.2.origin
ca3da13d69ce07ed9b31314f2404f9c6a2235067406b545d7b00517258cee9d6	42/70	UDS:Trojan-Dropper.Linux.Agent.gen	Android.BankBot.Coper.4363
724a56172f40177da76242ee169ac336b63d5df85889368d1531f593b658606b	40/63	HEUR:Trojan-Banker.AndroidOS.Sova.i	Android.BankBot.992.origin
1a9a54eb8b3ed707852eace623b935bc687c46d5e0cfa793751e778dacd8b62b	40/67	HEUR:Backdoor.Linux.Mirai.ld	Android.Siggen.Susp2.4391
95a1793b5bd345b18eda377e529d74f360590764bf454dee96575e2f31706723	40/64	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.Spy.4521
a778818cc08d213d1f66db5fa8b34aabc862569ecb99fd1aea37ff6ccab48400	39/67	HEUR:Trojan-Banker.AndroidOS.Bian.f	Android.BankBot.9305
47bb7f855cdf116c62499240089fa1b7a69585e8b7f639e192b9d038da4094cd	39/66	HEUR:Trojan-Banker.AndroidOS.Agent.io	Android.BankBot.11270
153410238d01773e5c705c6d18955793bd61cb2e82c5c7656e74563bb43b3ffa	39/68	HEUR:Trojan-Dropper.AndroidOS.Hqwar.ia	Android.BankBot.14812
376d13affcbfc5d5358d39aba16b814f711f2e81632059a4bce5af060e038ea4	39/66	HEUR:Trojan-Dropper.AndroidOS.Hqwar.hr	Android.BankBot.14551
cde47bed5ec83e29f6455ec35228c3f56d488393668f5b720b79fcd6cb3bc098	39/67	HEUR:Trojan-Spy.AndroidOS.SpyNote.bo	Android.Hidden.1.origin

ddabe5999dbc9ffbc8b804a7ea6bb61415bc88e346b2d2c0d53bfff9dcc88d32	39/68	HEUR:Trojan-Dropper.AndroidOS.Hqwar.bk	Android.BankBot.11270
1c273110f3aec84e76b13dff60521cc65731ef83657240f09cc9f5e0b1926c32	39/62	HEUR:Trojan.AndroidOS.Piom.aiuj	Android.SmsSpy.10966
04b4f6aefde0a647337e2000dfc18ae5c1b973bbe57571322eed408fb0ecee0a	39/64	HEUR:Trojan-Spy.AndroidOS.SmsThief.tt	Android.SmsSpy.7189
360fbf0eef153c745ca790d7e2222f1c0ea6e3edcbd42fb16d30b2f24e4dc402	39/67	HEUR:Trojan-Spy.AndroidOS.Agent.zv	Android.SmsSpy.830.origin
f4d7e9ec4eda034c29b8d73d479084658858f56e67909c2ffedf9223d7ca9bd2	38/68	HEUR:Trojan-Spy.AndroidOS.Agent.aco	Android.BankBot.847.origin
d62705186c488bb26fccdb1404931223a887004fd6704ac1483e599a15e92792	38/67	HEUR:Trojan-Spy.AndroidOS.Agent.acj	Android.Backdoor.690.origin
4ed2c0888048def2441087d03d847584c0f130c10e98d3978b80e4af8ef19fb7	38/67	HEUR:Trojan-Spy.AndroidOS.Agent.aho	Android.SmsSpy.10966
df3d18f6b2ee85551b42324e59de7488afa9d459819a2e9cefe3e5ce5322939f	38/67	HEUR:Trojan-Spy.AndroidOS.Agent.aho	Android.SmsSpy.10966
59cedb5ea89e38810819f149a4b9635866350d59e010c3133f8dda041577c9d7	38/67	HEUR:Trojan-Spy.AndroidOS.Agent.zv	Android.SmsSpy.830.origin
cb47e96d5c90d32dc9a55149e53fcc88b723a193bb5bf6b57db1d7cdef1d163c	38/64	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.SmsSpy.11058
4f59c653be63b76afef598868785659dff0e3fa320884008fb6bb2e5639aa5bd	38/66	HEUR:Trojan.AndroidOS.Boogr.gsh	Android.SmsSpy.830.origin
73ec42013f549f3b709852770c9e2769a56943173d304f939f6bcdf567cda4f1	38/65	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.Spy.989.origin
39ccc5d7009aefac18891c70a16b857b98481d6f69800075e31900c4dc9ef000	37/66	HEUR:Trojan-Dropper.AndroidOS.Hqwar.hr	Android.BankBot.14902
1430dc0cc26adfb7b7715624602767af26f453e3f34df66f1a6d41c38a35819c	37/67	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.Spy.989.origin
1f333d910158316c6211f7b24d118aadca778abb3964f3a4b7d3914e49278b7d	37/66	HEUR:Trojan-Spy.AndroidOS.SpyNote.dn	Android.SpyMax.72.origin
71c78101f7792fe879a082e323fed89c5e4a43132d01d3f79ed02afd8db45497	37/67	HEUR:Trojan-Downloader.AndroidOS.Banker.j	Android.DownLoader.1069.origin
e3fc59d0c60dfdcc64778b39d70635db1824254206dcc78a38c77b66a6c99b33	37/66	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.Spy.4521
5b0b1c7fc6e0677866a01483654d286ead9622dd3faf362133e28051bcf7989e	37/67	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.Spy.4521
fdd19235e6886e2b4386a91852bb8bd464bbb08097d4bf4927c0a65a84276014	37/64	HEUR:Trojan.AndroidOS.Boogr.gsh	Android.SmsSpy.830.origin
ca0d2a1f8081e66130801082ced73f04b8ac4f33adf6de58d4ca2a84bb6d799b	37/65	HEUR:Trojan-Spy.AndroidOS.Agent.zp	Android.Spy.917.origin
1bd1533dc9b6dde06ada49109d583dac49098ee232e181f727c26fcce658b89b	37/65	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.Spy.989.origin
2152b98a832ce3e442b486ceeaa8e41a8a99657168986757913e2d916c7b1c21	36/68	HEUR:Backdoor.AndroidOS.Agent.il	Android.Mirai.7.origin
29926c0101734a0c34efd310d58d055f266e28960c65f00f905327dcbb186711	36/70	HEUR:Trojan.AndroidOS.Harly.at	
77ffe8b7b3efa39fa35daaee1f70f560079c96c680f9d9d85997d658e2b367a9	36/66	HEUR:Trojan-Downloader.AndroidOS.Masplot.p	Android.RemoteCode.6833
4e9e6b28830190b0ee3655b2a1c348ba037ef584c66c14dcca929cfc53233e6b	36/67	HEUR:Trojan-Spy.AndroidOS.Agent.aho	Android.SmsSpy.10966
3d0985b998595dbcf3f136765f362da9c6033e42bc3388d2a5d708ac3b2dcf70	36/67	HEUR:Backdoor.AndroidOS.Basdoor.c	Android.Spy.4521
4c96a751c1ba60c3d28a9413d71166401c796d1acb3d5f93d4fee20db7984344	36/65	HEUR:Trojan.AndroidOS.Piom.aiuj	Android.SmsSpy.10966
cb0dd3d4234c701c0dc146f557ad3f77af82cfa9f5d7f4518bd358951b3b8d8e	36/67	HEUR:Trojan.AndroidOS.Boogr.gsh	Android.SmsSpy.830.origin
824bc8f232feb2eefdcdda66dd56baf2eb654ede2aa3f4ce2351da26900a05ca	36/67	HEUR:Trojan-Spy.AndroidOS.Realrat.l	Android.Spy.989.origin

Приложение Б. Состав материалов

В каждой папке задания лежит `README.md` с описанием файлов и командами проверки.

ФАЙЛ	СОДЕРЖИМОЕ
`task1_apk_hashes/collect.py`	Скрипт сбора подборки (MalwareBazaar + VirusTotal), кэш и бюджет запросов
`task1_apk_hashes/out/apk_verdicts.csv`	2000 строк: MD5, SHA-1, SHA-256, семейство, вердикты вендоров MalwareBazaar и 24 ключевых AV-движков, вредоносные вложенные файлы
`task1_apk_hashes/out/apk_verdicts_long.csv`	16 772 строки «SHA-256 — источник — вендор — вердикт», все вендоры
`task1_apk_hashes/out/apk_inner_files.csv`	196 вредоносных файлов внутри APK: имя, SHA-256, тип, число детектов, примеры вердиктов
`task1_apk_hashes/out/apk_verdicts.json`	Все собранные данные, включая Androguard-метаданные
`task1_apk_hashes/out/REPORT.md`	Автоматическая сводка по подборке
`task1_apk_hashes/cache/`	Сырые ответы API, из которых `--report-only` побайтово воспроизводит `out/`
`task2_sigtool/sigtool.py`	Генератор байтовых сигнатур
`task2_sigtool/test_sigtool.py`	8 модульных тестов

Запуск

```
cd task1_apk_hashes
export MB_AUTH_KEY=… VT_API_KEY=…
python3 collect.py --mb-limit 2000 --vt-files 300 --vt-bundled 150
python3 collect.py --report-only        # пересобрать выгрузку из кэша

cd ../task2_sigtool
python3 -m unittest -v test_sigtool
python3 sigtool.py --g1 G1/ --g2 G2/ --unpack -f json -o sigs.json
```

Источники

- MalwareBazaar API — abuse.ch
- VirusTotal API v3

> **Безопасность.** Образцы ВПО в работе не скачивались и не запускались: задание 1 выполнено только по метаданным из API, задание 2 проверено на синтетических данных.

Приложение B. Использование ИИ

При выполнении заданий использовался ИИ-ассистент:

- **Код.** Помощь в написании скрипта сбора `collect.py`, прототипа `sigtool.py` и тестов к нему.
- **Оформление.** Структура и вёрстка отчёта, схемы, таблицы и графики по готовым данным.
- **Уточнение фактов.** Сверка цифр и хешей в тексте с выгрузками и выводом инструментов.

Постановка подхода, проверка результатов и выводы выполнены автором.